

TourGuide

TourGuide

Documentation fonctionnelle et technique

Version 2.0, application refactorisée
18 octobre 2026
Auteur : GHOUDI Saber
Dépôt : https://github.com/djerbiano/JavaPathENProject8

Sommaire

1. **Présentation du projet**
   - **1.1 Objectifs du projet**
   - **1.2 Hors du champ d'application**
   - **1.3 Mesures du projet**
2. **Spécifications fonctionnelles**
3. **Spécifications techniques**
   - **3.1 Schémas de conception technique**
   - **3.2 Glossaire**
   - **3.3 Solutions techniques**
   - **3.4 Autres solutions non retenues**

1. Présentation du projet

TourGuide est une application **Spring Boot** qui accompagne les voyageurs pendant leurs déplacements. À partir de la position GPS de l'utilisateur, elle :

- **recommande les 5 attractions touristiques les plus proches** ;
- **attribue des points de récompense** quand l'utilisateur visite une attraction ;
- **propose des offres de voyage** d'agences partenaires, adaptées à ses préférences (durée, nombre d'adultes et d'enfants).

Public cible : les voyageurs, via les applications clientes qui consomment l'API REST de TourGuide, et indirectement les agences de voyage partenaires dont les offres sont mises en avant.

Avantages commerciaux :

- **fidélisation** : les points de récompense incitent les utilisateurs à revenir et à visiter de nouvelles attractions ;
- **ventes** : les offres personnalisées génèrent du chiffre d'affaires auprès des agences partenaires ;
- **croissance** : l'application refactorisée supporte **100 000 utilisateurs**, alors que la version initiale était déjà saturée.

1.1 Objectifs du projet

TourGuide est passée de quelques centaines à **30 000 utilisateurs par jour**, avec un objectif de **100 000**. La version initiale était devenue **trop lente** : il fallait plus de 2 heures pour localiser 100 000 utilisateurs et près de 18 heures pour calculer leurs récompenses. Elle présentait aussi des **tests en échec intermittent** et une **recommandation d'attractions incorrecte**.

Le projet a pour objectifs de :

1. **corriger les dysfonctionnements** : tests intermittents dus à un problème de concurrence, recommandation des attractions ;
2. **atteindre les objectifs de performance** pour 100 000 utilisateurs ;
3. **sécuriser les évolutions futures** avec un pipeline d'intégration continue.

1.2 Hors du champ d'application

Envisagé	Raison de l'exclusion
Déploiement en production	Non demandé pour cette version. Le pipeline s'arrête à la construction du jar.
Base de données persistante	Les utilisateurs sont simulés en mémoire (mode test, `InternalTestHelper`). La persistance relève d'une version ultérieure.
Authentification et sécurité de l'API	Hors du périmètre de la mission, centrée sur la correction et la performance.
Restructuration complète en couches (packages `controller`, `repository`…)	Envisagée puis écartée, pour concentrer l'effort sur les objectifs prioritaires et garder la structure d'origine.
Interface utilisateur (front-end)	TourGuide expose uniquement une API REST.

1.3 Mesures du projet

Le succès du projet se mesure avec des **critères objectifs et vérifiables**, contrôlés automatiquement par les tests et par le pipeline :

Indicateur	Objectif	Résultat obtenu
Tests automatisés	100 % réussis, aucun test désactivé	**Atteint** : 12/12 réussis, 0 ignoré
Stabilité des tests liés à la concurrence	Aucun échec intermittent	**Atteint** : `nearAllAttractions` réussi à chaque exécution
Recommandation d'attractions	Toujours 5 attractions, triées par distance	**Atteint** : testé et vérifié sur `/getNearbyAttractions`
Localisation de 100 000 utilisateurs	≤ 15 min (900 s)	**Atteint** : **384 s** (initialement 7 579 s), Tracker actif
Récompenses de 100 000 utilisateurs	≤ 20 min (1 200 s)	**Atteint** : **518 s** (initialement 64 020 s)
Intégration continue	Compilation, tests et jar automatiques à chaque push	**Atteint** : pipeline GitHub Actions vert

Le détail des mesures pour chaque palier (100 à 100 000 utilisateurs) figure dans le document de mesure de performance (livrable 3).

Note sur la mesure de localisation. Le `Tracker` démarre dès la création de `TourGuideService` et localise lui aussi les 100 000 utilisateurs, avec le **même pool de threads** que le test. Le test mesure donc 100 000 localisations pendant que 100 000 autres sont en cours : **384 s**, soit environ 200 000 localisations traitées. Ce chiffre est plus réaliste qu'une mesure isolée, car en production le Tracker tourne en permanence. Mesurée sans le Tracker concurrent, la localisation prenait 201s.

2. Spécifications fonctionnelles

TourGuide expose une **API REST**. Chaque fonctionnalité correspond à un endpoint `GET` qui reçoit le nom de l'utilisateur en paramètre (`?userName=...`) et renvoie une réponse au format JSON.

2.1 Liste des fonctionnalités

Fonctionnalité	Endpoint	Description
Vérifier que l'application répond	`GET /`	Renvoie un message d'accueil. Sert à contrôler que le service est démarré.
Localiser un utilisateur	`GET /getLocation`	Renvoie la dernière position connue de l'utilisateur (latitude, longitude, date). Si aucune position n'est connue, l'application interroge le service GPS.
Recommander les attractions proches	`GET /getNearbyAttractions`	Renvoie les **5 attractions les plus proches** de la dernière position de l'utilisateur, avec la distance et les points de récompense de chacune.
Consulter ses récompenses	`GET /getRewards`	Renvoie la liste des récompenses gagnées : attraction visitée, position de la visite et points obtenus.
Obtenir des offres de voyage	`GET /getTripDeals`	Renvoie des offres d'agences partenaires, calculées selon les préférences de l'utilisateur et ses points de récompense cumulés.
Suivi automatique des utilisateurs	aucun (tâche de fond)	Toutes les **5 minutes**, l'application localise tous les utilisateurs et met à jour leurs récompenses, sans action de leur part.

2.2 Règles de gestion

Règle	Détail
Attractions recommandées	Toujours **5 attractions**, triées de la plus proche à la plus éloignée, **quelle que soit la distance**. Un utilisateur éloigné de toutes les attractions reçoit quand même 5 recommandations.
Gain d'une récompense	Une attraction rapporte des points si l'utilisateur est passé à **moins de 10 miles** de celle-ci (paramètre `proximityBuffer`).
Unicité des récompenses	Une attraction ne rapporte des points **qu'une seule fois** par utilisateur, même s'il la visite plusieurs fois.
Points de récompense	Le nombre de points est fourni par le service partenaire **RewardCentral**, pour chaque couple attraction / utilisateur.

Règle	Détail
Offres de voyage	Calculées par le service **TripPricer** à partir des préférences (durée du séjour, nombre d'adultes et d'enfants) et du **total des points de récompense**.
Distances	Exprimées en **miles terrestres**, calculées à partir des coordonnées GPS (latitude, longitude).

2.3 Exemple : recommandation des attractions

Requête : `GET /getNearbyAttractions?userName=internalUser0`

Réponse (extrait, 1 élément sur 5) :

```
[
  {
    "attractionName": "Disneyland",
    "attractionLatitude": 33.817595,
    "attractionLongitude": -117.922008,
    "userLatitude": 33.912,
    "userLongitude": -118.003,
    "distance": 7.94,
    "rewardPoints": 412
  },
  ...
]
```

Figure 1 : format de réponse de /getNearbyAttractions. Les valeurs sont illustratives.

Chaque élément contient les 7 informations demandées : le nom de l'attraction, ses coordonnées, celles de l'utilisateur, la distance qui les sépare (en miles) et les points de récompense que rapporterait la visite.

2.4 Exigences non fonctionnelles

Exigence	Cible	Résultat
Localiser 100 000 utilisateurs	≤ 15 minutes	**384 s**
Calculer les récompenses de 100 000 utilisateurs	≤ 20 minutes	**518 s**
Fiabilité en accès concurrent	Aucune erreur ni doublon quand plusieurs traitements touchent le même utilisateur	**Atteint** (voir 3.3)
Qualité continue	Compilation, tests et jar vérifiés à chaque modification	**Atteint** (pipeline CI)

3. Spécifications techniques

3.1 Schémas de conception technique

Les deux schémas ci-dessous décrivent l'application refactorisée sous deux angles complémentaires : l'**architecture** (qui parle à qui) et le **déroulement d'un traitement parallèle** (diagramme de séquence UML). Les éléments ajoutés ou modifiés en version 2 sont signalés **en vert** sur chaque schéma.

3.1.1 Architecture technique (figure 2, page suivante)

Ce schéma met à jour le schéma d'architecture fourni au démarrage du projet. Il se lit **de haut en bas**, en suivant une requête :

1. Un **client HTTP** (application cliente, navigateur, Postman) envoie une requête `GET` à l'application et reçoit une réponse JSON.
2. **TourGuideController** reçoit la requête. Il ne contient aucune logique : il délègue à **TourGuideService**.
3. **TourGuideService** porte la logique métier : localisation, recommandation des attractions, offres de voyage. Il appelle les librairies externes **gpsUtil** et **TripPricer**, et délègue le calcul des points à **RewardsService**.
4. **RewardsService** calcule les récompenses : il compare les positions visitées aux attractions (fournies par gpsUtil) et obtient les points auprès de **RewardCentral**.

Ce qui change en version 2 :

- **Deux pools de 100 threads** (en vert) : un dans TourGuideService pour localiser les utilisateurs, un dans RewardsService pour calculer leurs récompenses. Ils permettent de traiter de nombreux utilisateurs **en parallèle** au lieu de les traiter un par un.
- Le **Tracker**, thread de fond qui met à jour tous les utilisateurs toutes les 5 minutes (lui aussi en parallèle, via `trackAllUsersLocation`), et les **données en mémoire** (listes thread-safe `CopyOnWriteArrayList`) apparaissent explicitement, car ils sont au cœur des problèmes de concurrence corrigés.
- **TourGuideModule** ne fournit plus que les beans des librairies externes (`GpsUtil`, `RewardCentral`). Le doublon du bean `RewardsService` a été supprimé.
- L'**intégration continue** (bandeau du bas) vérifie automatiquement chaque modification.

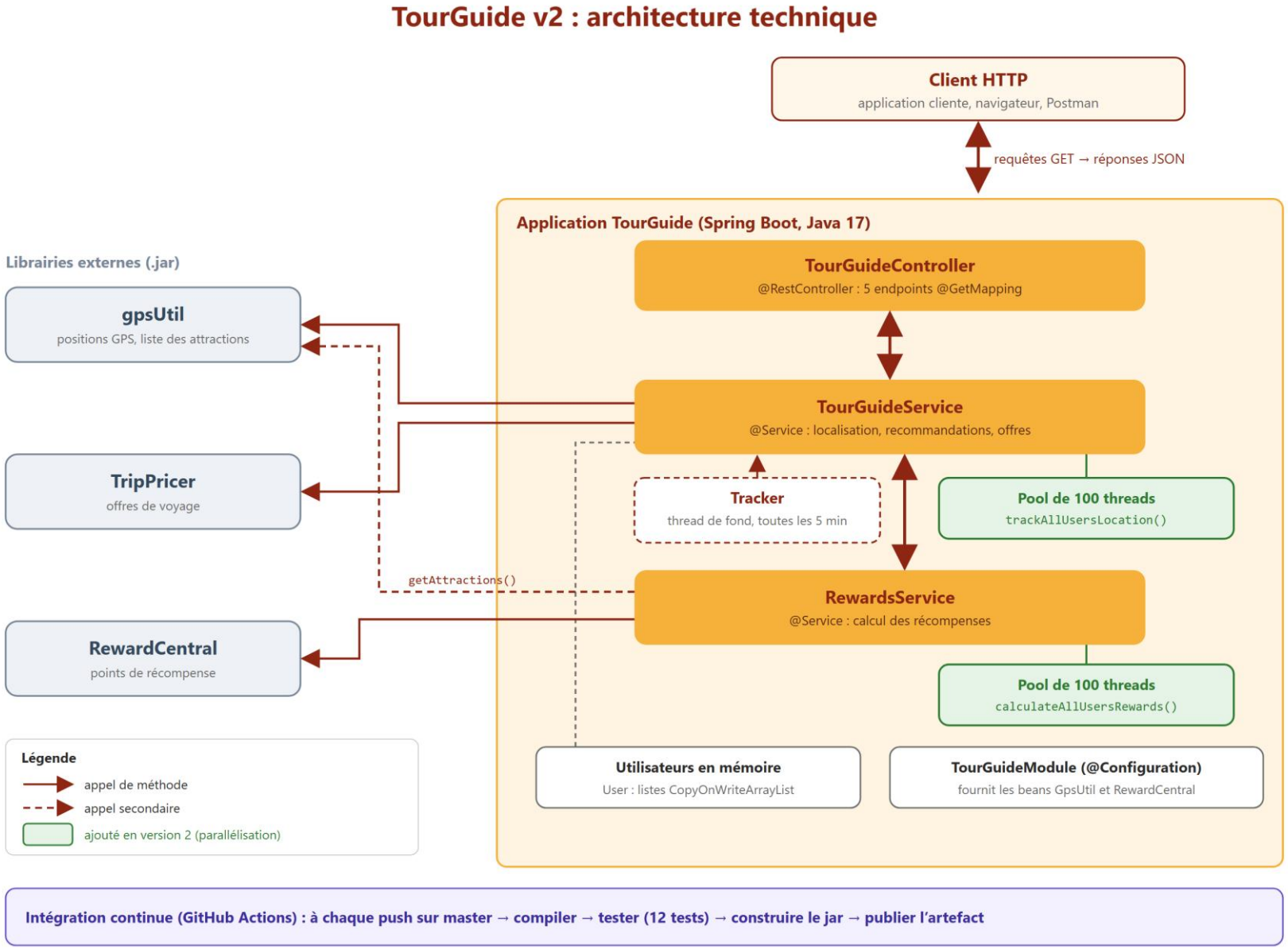


Figure 2 : architecture technique de TourGuide v2. Les flèches pleines sont des appels de méthode ; en vert, les pools de threads ajoutés en version 2

3.1.2 Diagramme de séquence UML (figure 3, page suivante)

Le diagramme de séquence montre, dans l'ordre chronologique, comment `trackAllUsersLocation` localise tous les utilisateurs en parallèle. C'est le mécanisme qui a fait passer la localisation de 100 000 utilisateurs de **7 579 s à 384 s**. Le calcul des récompenses (`calculateAllUsersRewards`) suit exactement le même schéma, avec `runAsync` à la place de `supplyAsync`.

1. **Lancement (cadre `loop`)** : pour chaque utilisateur, `TourGuideService` confie une tâche au pool avec `CompletableFuture.supplyAsync(tâche, pool)` et reçoit immédiatement une promesse (`CompletableFuture`). Cette boucle est très rapide : elle ne fait que remplir la file d'attente.
2. **Exécution (cadre `par`)** : les 100 threads du pool exécutent les tâches en même temps. Chaque tâche appelle `trackUserLocation` : position GPS via gpsUtil, puis calcul des récompenses via RewardsService et RewardCentral. Pendant qu'un thread attend un service externe, les 99 autres avancent.
3. **Attente de la fin** : `CompletableFuture.allOf(...).join()` bloque l'appelant jusqu'à ce que **toutes** les tâches soient terminées. La méthode ne rend donc la main qu'une fois tous les utilisateurs localisés.

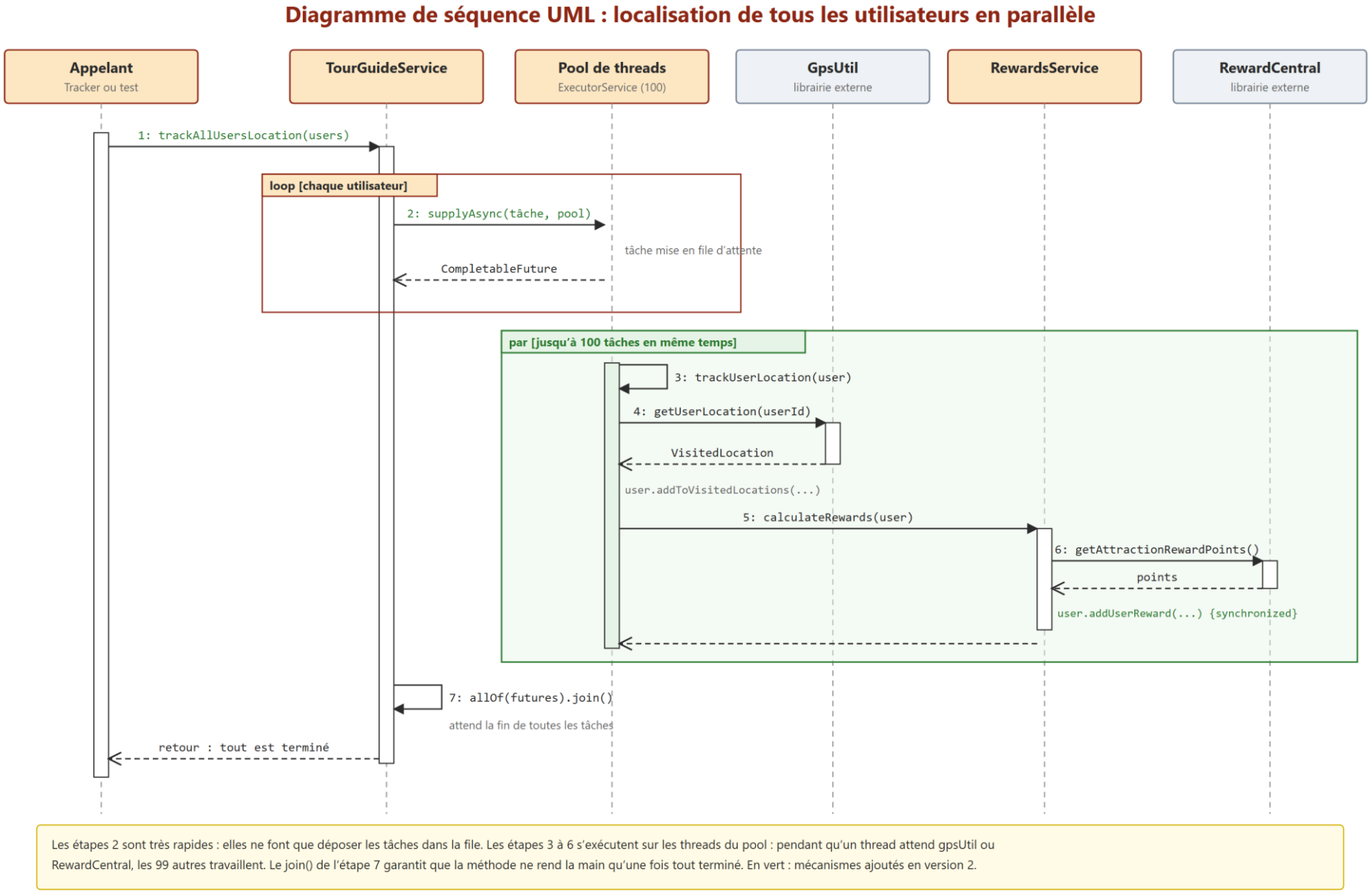


Figure 3 : diagramme de séquence UML de trackAllUsersLocation : les tâches sont déposées dans la file (loop), puis exécutées en parallèle par le pool (par)

3.2 Glossaire

Les termes sont classés en deux groupes : le vocabulaire **métier** de TourGuide, puis le vocabulaire **technique** utilisé dans ce document.

Vocabulaire métier

Terme	Définition
Attraction	Lieu touristique (parc, musée…) avec un nom et des coordonnées GPS. Fournie par gpsUtil.
Position visitée (`VisitedLocation`)	Position GPS d'un utilisateur à un instant donné : identifiant de l'utilisateur, latitude, longitude, date.
Récompense (`UserReward`)	Points gagnés par un utilisateur pour être passé près d'une attraction. Une seule par attraction.
Points de récompense	Nombre de points attribué par RewardCentral pour un couple attraction / utilisateur.
Préférences utilisateur	Durée du séjour, nombre d'adultes et d'enfants, utilisés pour calculer les offres de voyage.
Offre de voyage (`Provider`)	Proposition d'une agence partenaire (nom, prix), calculée par TripPricer.
Tracker	Tâche de fond qui localise tous les utilisateurs et met à jour leurs récompenses toutes les 5 minutes.
Mile terrestre	Unité de distance utilisée par l'application (1 mile ≈ 1,609 km).

Vocabulaire technique

Terme	Définition
API REST	Interface qui expose des fonctionnalités via des URL et des méthodes HTTP (ici `GET`), avec des réponses au format JSON.
Endpoint	URL précise de l'API, par exemple `/getNearbyAttractions`.
DTO (Data Transfer Object)	Objet simple qui ne sert qu'à transporter des données vers le client, ici `NearbyAttractionDTO`.
Bean Spring	Objet créé et géré par Spring. Par défaut, il n'en existe qu'un par classe (singleton).
Injection de dépendances	Spring fournit à un objet ce dont il a besoin, ici par son **constructeur**.
Thread	Fil d'exécution : une suite d'instructions qui s'exécute en parallèle d'autres.
Pool de threads (`ExecutorService`)	Groupe fixe de threads réutilisables à qui l'on confie des tâches. Les tâches en trop attendent dans une file.

Terme	Définition
`CompletableFuture`	Promesse d'un résultat qui arrivera plus tard. `supplyAsync` lance une tâche qui renvoie un résultat, `runAsync` une tâche sans résultat.
`allOf(...).join()`	`allOf` combine plusieurs promesses en une seule ; `join()` attend qu'elle soit terminée.
Condition de course	Bug dont le résultat dépend de l'ordre d'exécution des threads. Il est souvent intermittent.
Thread-safe	Utilisable par plusieurs threads en même temps sans erreur ni corruption des données.
`CopyOnWriteArrayList`	Liste thread-safe : chaque modification crée une copie, donc un parcours n'est jamais perturbé par un ajout.
`synchronized`	Mot-clé Java : un seul thread à la fois peut exécuter la méthode sur un même objet. L'opération devient atomique.
`@PreDestroy`	Annotation : méthode appelée par Spring juste avant de détruire un bean, à l'arrêt de l'application.
Shutdown hook	Code enregistré auprès de la JVM et exécuté à l'arrêt du programme.
Test intermittent	Test qui échoue parfois et réussit parfois, sans modification du code. C'est souvent le signe d'un problème de concurrence.
Intégration continue (CI)	Vérification automatique du code (compilation, tests, construction) à chaque modification poussée.
Workflow GitHub Actions	Fichier YAML (`.github/workflows/ci.yml`) qui décrit les étapes du pipeline CI.
Artefact	Fichier produit et conservé par le pipeline, ici le jar de l'application.

3.3 Solutions techniques

Technologies : Java 17, Spring Boot 3.1.1, Maven, JUnit 5, GitHub Actions. Les librairies `gpsUtil`, `RewardCentral` et `TripPricer` sont fournies sous forme de `.jar` et utilisées sans modification.

Chaque besoin est présenté de la même façon : le **problème** constaté, la **solution** retenue et le **résultat** mesuré.

3.3.1 Tests en échec intermittent (problème de concurrence)

- **Problème** : le test `nearAllAttractions` échouait de façon aléatoire avec une `ConcurrentModificationException`. Cause : le `Tracker`, lancé dès la création de `TourGuideService`, modifiait les listes d'un `User` pendant que le test les parcourait. Ces listes étaient des `ArrayList`, non thread-safe. Un second bug faisait aussi échouer le test : `addUserReward` comparait un nom (`String`) à un objet `Attraction`, donc la condition était toujours fausse.
- **Solution** : les listes `visitedLocations` et `userRewards` deviennent des

`CopyOnWriteArrayList`, adaptées à ce cas où l'on lit souvent et où l'on écrit peu. La condition de `addUserReward` compare maintenant des noms d'attraction, et la méthode est `synchronized` pour que la vérification et l'ajout soient atomiques (pas de doublon).

- **Résultat** : le test passe à chaque exécution, sans que son code ait été modifié, comme l'exige l'énoncé.

3.3.2 Recommandation des 5 attractions les plus proches

- **Problème** : `getNearByAttractions` filtrait les attractions dans un rayon de 200 miles. Le nombre de résultats variait (0, 1, plusieurs) au lieu d'être toujours 5.
- **Solution** : on trie les attractions par distance croissante, puis on garde les 5 premières (`sorted(Comparator.comparingDouble(...))` et `limit(5)`). La réponse de l'API passe par un DTO, `NearbyAttractionDTO`, qui contient exactement les 7 informations demandées. Les points de récompense sont obtenus via `RewardsService.getRewardPoints`, qui reste le seul point d'accès à RewardCentral.
- **Résultat** : toujours 5 attractions, triées par distance, quelle que soit la position de l'utilisateur. C'est vérifié par un test ajouté (`getNearByAttractionsDetails`) et par un appel de l'endpoint dans le navigateur.

3.3.3 Performance de la localisation (gpsUtil)

- **Problème** : les utilisateurs étaient localisés un par un. Chaque appel à gpsUtil attend une réponse (service distant simulé), et ces attentes s'additionnaient : 7 579 s pour 100 000 utilisateurs.
- **Solution** : comme le temps est passé à **attendre** et non à calculer, on traite beaucoup d'utilisateurs en même temps. `TourGuideService` dispose d'un pool de 100 threads (`Executors.newFixedThreadPool(100)`). `trackUserLocationAsync` confie chaque utilisateur au pool avec `CompletableFuture.supplyAsync`, et `trackAllUsersLocation` attend la fin de toutes les tâches avec `CompletableFuture.allOf(...).join()`. Le `Tracker` utilise aussi cette méthode. Le pool est fermé à l'arrêt de l'application dans le shutdown hook.
- **Résultat** : **384 s** pour 100 000 utilisateurs, avec le Tracker actif en parallèle (limite : 900 s).

```
public void trackAllUsersLocation(List<User> users) {
    List<CompletableFuture<VisitedLocation>> futures = users.stream()
            .map(user -> trackUserLocationAsync(user))
            .collect(Collectors.toList());
    CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
}
```

3.3.4 Performance des récompenses (RewardCentral)

- **Problème** : même cause que pour la localisation. Chaque appel à RewardCentral attend une réponse : 64 020 s pour 100 000 utilisateurs.
- **Solution** : même principe, appliqué à `RewardsService`, qui a son propre pool de 100

threads. `calculateRewardsAsync` utilise `CompletableFuture.runAsync`, car `calculateRewards` ne renvoie rien. `calculateAllUsersRewards` attend la fin de toutes les tâches. Le pool est fermé par une méthode `shutdown()` annotée `@PreDestroy`, appelée par Spring à l'arrêt.

- **Résultat** : **518 s** pour 100 000 utilisateurs (limite : 1 200 s).

3.3.5 Qualité du code et annotations

- `@RequestMapping` remplacé par `@GetMapping` sur les 5 endpoints : ils ne font que lire des données et n'acceptent plus que la méthode `GET`.
- Injection par **constructeur** dans `TourGuideController` (champ `private final`), au lieu de l'injection par champ.
- Suppression du bean `RewardsService` déclaré en double (`@Service` et `@Bean`), qui créait un second pool de threads inutile.
- Suppression des `@Disabled` laissés en commentaire, du TODO traité et des imports inutilisés.

3.3.6 Stratégie de tests

Tests	Rôle et justification
`TestRewardsService`, `TestTourGuideService`	Tests unitaires des fonctionnalités : récompenses, localisation, recommandations, offres. `nearAllAttractions` est réactivé ; `getNearbyAttractions` est réactivé ; `getNearByAttractionsDetails` est ajouté pour vérifier les 5 DTO triés par distance.
`TestPerformance`	Tests de charge : localisation et récompenses de 100 000 utilisateurs, avec une limite de temps. Ils sont lancés **progressivement** (100 → 1 000 → … → 100 000) pour suivre l'évolution des temps. Conformément à l'énoncé, ils ne font pas eux-mêmes de multi-threading : ils appellent les méthodes du service.
`TourguideApplicationTests`	Vérifie que le contexte Spring démarre, donc que tous les beans et injections sont corrects.

Résultat : 12 tests, 0 échec, 0 ignoré.

3.3.7 Pipeline d'intégration continue

- **Outil** : GitHub Actions, car le dépôt est déjà sur GitHub. Il n'y a rien à installer ni à héberger.
- **Structure** (`.github/workflows/ci.yml`), déclenchée à chaque push et pull request sur `master` : récupération du code → installation de Java 17 → installation des 3 librairies locales (`mvn install:install-file`) → **compilation** → **tests** → **construction du jar** → publication du jar comme artefact.

- **Démarche** : le pipeline a d'abord été validé sans les tests de performance (`-Dtest=!TestPerformance`, environ 1 minute), puis ceux-ci ont été réactivés (environ 12 minutes), comme le conseille l'énoncé.

3.4 Autres solutions non retenues

Solution envisagée	Pourquoi elle n'a pas été retenue
Modifier le test `nearAllAttractions` pour qu'il passe	Interdit par l'énoncé : il faut corriger le code de l'application, pas le test. Cela aurait aussi masqué le vrai bug.
Garder le filtre de rayon (200 miles) en l'élargissant	Ne garantit jamais exactement 5 résultats : le nombre dépend toujours de la position de l'utilisateur.
`supplyAsync` sans pool dédié	Même pool par défaut (`ForkJoinPool.commonPool`), avec environ autant de threads que de cœurs : beaucoup trop peu pour des tâches qui passent leur temps à attendre.
Un thread par utilisateur	100 000 threads consommeraient trop de mémoire (environ 1 Mo chacun) et feraient planter l'application.
GitLab CI ou Jenkins	GitLab aurait demandé de migrer le dépôt ; Jenkins, d'installer et d'héberger un serveur. GitHub Actions est intégré au dépôt existant.